

## Step 3 — The Writer Agent Synthesizes a Final Report — Solution

---

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume Step 2's `state.py`, `supervisor.py`, `notes_agent.py`, `web_agent.py`, and `graph.py` already work.

### Basic Version

---

#### Approach 1 — the direct way, one prompt string

```
# writer_agent.py
from openai import OpenAI

client = OpenAI()

def run_writer(notes_findings, web_findings):
    prompt = (
        "Combine these findings into one short report:\n\n"
        f"Notes: {notes_findings}\n\n"
        f"Web: {web_findings}"
    )
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content
```

This works, and is a real plain LLM call with no tools. Its instruction ("combine these findings") is vague enough that the model will often play it safe and write one paragraph per source instead of truly merging them — that's exactly the gap Intermediate's more specific prompt closes.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

### Approach 1 — a specific synthesis instruction, and a node wrapper

```
# writer_agent.py
from openai import OpenAI
from langgraph.graph import END
from langgraph.types import Command
from state import TeamState

client = OpenAI()

WRITER_PROMPT = """You are writing a short status report from research findings.

Notes findings:
{notes_findings}

Web findings:
{web_findings}

Write one short, clear report. State what is known, citing which source(s)
support each point. Do not write one paragraph per source -- organize by
topic or conclusion instead."""

def run_writer(notes_findings: str, web_findings: str) -> str:
    prompt = WRITER_PROMPT.format(
        notes_findings=notes_findings or "(not checked for this request)",
        web_findings=web_findings or "(not checked for this request)",
    )
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def writer_agent_node(state: TeamState) -> Command:
    report = run_writer(state.get("notes_findings", ""), state.get("web_findings", ""))
    return Command(goto=END, update={"report": report})
```

```
# supervisor.py (updated routing)
from langgraph.graph import END
from langgraph.types import Command
from state import TeamState

def figure_out_needed(task: str) -> list[str]:
    needed = []
    task_lower = task.lower()
    if "note" in task_lower:
        needed.append("notes_agent")
    if "http" in task_lower or "fetch" in task_lower or "page" in task_lower:
        needed.append("web_agent")
    if not needed:
        needed.append("notes_agent")
    return needed

def supervisor_node(state: TeamState) -> Command:
    needed = state.get("needed") or figure_out_needed(state["task"])
    visited = state.get("visited") or []

    for specialist in needed:
        if specialist not in visited:
            return Command(goto=specialist, update={"needed": needed})

    return Command(goto="writer_agent")
```

```
# state.py (updated)
from typing import TypedDict

class TeamState(TypedDict):
    task: str
    needed: list[str]
    visited: list[str]
    notes_findings: str
    web_findings: str
    report: str
```

```
# graph.py (adds the writer node)
from langgraph.graph import StateGraph
from state import TeamState
from supervisor import supervisor_node
from notes_agent import notes_agent_node
```

```

from web_agent import web_agent_node
from writer_agent import writer_agent_node

def build_graph():
    graph = StateGraph(TeamState)
    graph.add_node("supervisor", supervisor_node)
    graph.add_node("notes_agent", notes_agent_node)
    graph.add_node("web_agent", web_agent_node)
    graph.add_node("writer_agent", writer_agent_node)
    graph.set_entry_point("supervisor")
    return graph.compile()

```

**Expected output**, running task 3 through `main.py` and printing `final_state["report"]` : something like

```

Project Atlas is on track for a Q3 launch (per team notes) and is currently
reported as 80% complete (per the status page). The notes also flag that
legal sign-off is still pending, which isn't mentioned on the status page --
worth confirming before launch.

```

**Difference from Basic:** the prompt now gives a specific, testable instruction ("organize by topic, not by source," "cite which source(s) support each point") instead of the vague "combine these." The `or` "(not checked for this request)" fallback also means a single-specialist task (task 1 or 2) still produces a clean report instead of a prompt with an awkward empty section.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

### Approach 1 — testing the Writer directly against deliberately conflicting findings

```
# test_writer_synthesis.py
from writer_agent import run_writer

notes_findings = "Project Atlas notes say the launch is targeted for Q3, on sched
web_findings = "The status page reports Project Atlas is delayed to Q4 due to a s

report = run_writer(notes_findings, web_findings)
print(report)
```

**Expected output:** something that names the discrepancy honestly, e.g.:

```
Sources disagree on Project Atlas's timeline: internal notes say Q3, on
schedule, while the status page reports a delay to Q4 due to staffing.
This should be confirmed with the project owner before reporting a firm date.
```

If your prompt instead produces a report that confidently states just one date and drops the conflict, that's a sign the prompt needs to say explicitly what to do when sources disagree (add a line like: "If sources disagree, say so plainly instead of picking one silently").

### Approach 2 — single-specialist tasks produce a clean report, not an awkward gap

```
# test_writer_single_source.py
from writer_agent import run_writer

report = run_writer("Project Atlas notes say the launch is targeted for Q3.", "")
print(report)
```

**Expected output:** a report that discusses only the notes finding, without saying something confusing like "the web source found nothing" (which would misleadingly imply a web check happened and came up empty, when really the web agent was never asked).

```
Project Atlas is targeted for a Q3 launch, according to team notes.
```

### Approach 3 — the full team, run end to end through the graph

```
# main.py
from graph import build_graph

TASKS = [
    "What do my notes say about Project Atlas?",
    "Fetch https://intranet.example.com/atlas/status and tell me what it says.",
    "Check my notes for anything about Project Atlas, and also fetch its status p
]

if __name__ == "__main__":
    app = build_graph()
    for task in TASKS:
        print(f"\n=== Task: {task} ===")
        final_state = app.invoke({
            "task": task, "needed": [], "visited": [], "notes_findings": "", "web
        })
        print("Report:\n" + final_state["report"])
```

#### Expected output (abridged, task 3 only):

```
=== Task: Check my notes for anything about Project Atlas, and also fetch its sta
Report:
Project Atlas is on track for a Q3 launch (per notes) and 80% complete
(per the status page). Legal sign-off is still pending per notes -- worth
confirming before launch.
```

**Difference from Intermediate, and between these 3 Advanced approaches:** Approach 1 tests the Writer against inputs that genuinely conflict, which the happy-path tests so far never exercised — this is the same instinct as Step 1's "test a query with no match," applied here to the Writer's honesty under disagreement rather than a specialist's honesty under an empty result. Approach 2 confirms the single-source case reads cleanly, not just that it doesn't crash. Approach 3 is the full, real end-to-end proof — all specialists, real MCP connections, the real Supervisor routing, and the real Writer, together.

**Which one should you actually write?** Intermediate's `run_writer` and `writer_agent_node` are what you carry forward. Run Approach 1's conflicting-findings test once, by hand, and tighten your prompt's instruction for the disagreement case if it doesn't already handle it honestly — this is a cheap, high-value check, since a report that silently picks one of two conflicting facts is a genuinely worse failure than one that's just poorly organized. Approach 3's full run is your actual proof this step

is done; keep it as `main.py`'s default behavior going into Step 4, where it gets one more scenario added — a Web Agent that can't connect at all.